

§ 5.4 遍历

对二叉树的访问多可抽象为如下形式：按照事先约定的某种规则或次序，对节点各访问一次而且仅一次。与向量和列表等线性结构一样，二叉树的这类访问也统称为遍历（traversal）。同样地，遍历操作之于二叉树的意义，在于为许多相关算法的实现提供了通用框架和基本接口。从算法策略的角度看，这一过程也等效于将半线性的树形结构转换为线性结构。

不过，因为二叉树已经不再属于线性结构，故相对于向量和列表等序列结构，二叉树的遍历略显复杂。为此，以下首先从递归的角度，给出若干种典型的二叉树遍历次序的定义，并按照117页代码5.1和121页代码5.5所列接口，给出相应的递归式实现；然后，为了提高遍历算法的实际效率，再分别介绍各种遍历接口的迭代式实现。

5.4.1 递归式遍历

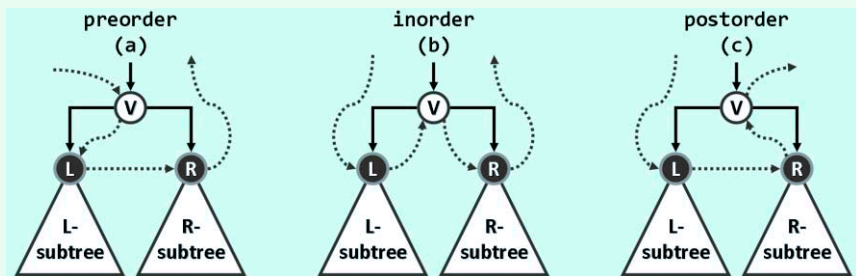


图5.14 二叉树遍历的全局次序由局部次序规则确定

二叉树本身并不具有天然的全局次序，故为实现遍历，需通过在各节点与其孩子之间约定某种局部次序，间接地定义某种全局次序。

按惯例左兄弟优先于右兄弟，故若将节点及其孩子分别记作V、L和R，则如图5.14所示，局部访问的次序可有VLR、LVR和LRV三种选择。根据节点V在其中的访问次序，三种策略也相应地分别称作先序遍历、中序遍历和后序遍历，分述如下。

■ 先序遍历

得益于递归定义的简洁性，如代码5.11所示，只需数行即可实现先序遍历算法。

```
1 template <typename T, typename VST> //元素类型、操作器
2 void travPre_R(BinNodePosi(T) x, VST& visit) { //二叉树先序遍历算法（递归版）
3     if (!x) return;
4     visit(x->data);
5     travPre_R(x->lChild, visit);
6     travPre_R(x->rChild, visit);
7 }
```

代码5.11 二叉树先序遍历算法（递归版）

为遍历（子）树x，首先核对x是否为空。若x为空，则直接退出——其效果相当于递归基。反之，若x非空，则按照先序遍历关于局部次序的定义，优先访问其根节点x；然后，依次深入左子树和右子树，递归地进行遍历。实际上，这一实现模式也同样可以应用于中序和后序遍历。

经过遍历之后,即可相应地在节点之间定义出某种线性次序,称作遍历(生成)序列。图5.15以一棵具体的二叉树为例,给出了完整的先序遍历过程以及生成的遍历序列。

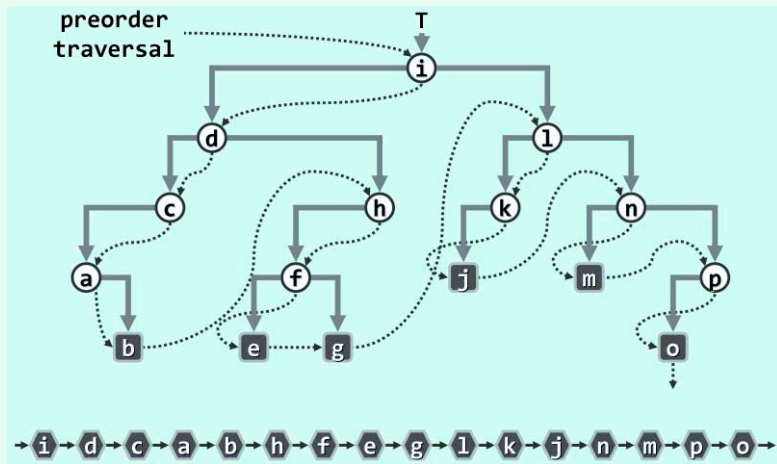


图5.15 二叉树先序遍历序列

后序遍历

```
1 template <typename T, typename VST> //元素类型、操作器
2 void travPost_R(BinNodePosi(T) x, VST& visit) { //二叉树后序遍历算法(递归版)
3     if (!x) return;
4     travPost_R(x->lChild, visit);
5     travPost_R(x->rChild, visit);
6     visit(x->data);
7 }
```

代码5.12 二叉树后序遍历算法(递归版)

仿照以上先序遍历的模式,可如代码5.12所示实现递归版后序遍历算法。

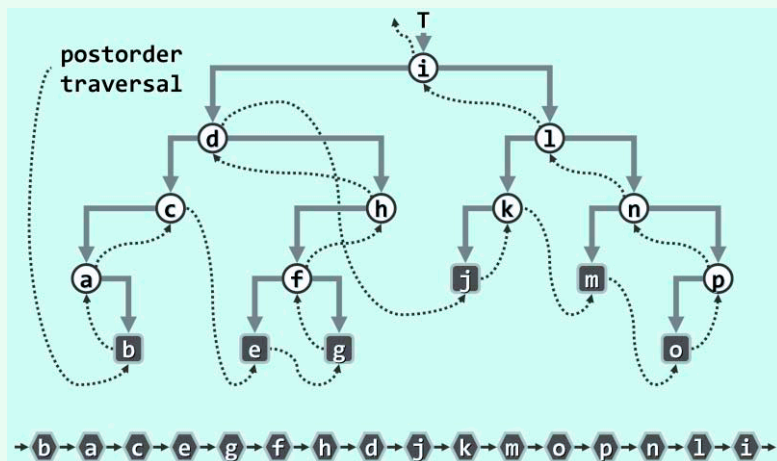


图5.16 二叉树的后序遍历序列

按照后序遍历规则，为遍历非空的（子）树 x ，将在依次递归遍历其左子树和右子树之后，才访问节点 x 。对于以上二叉树实例，其完整的后序遍历过程以及生成的遍历序列如图5.16所示。与图5.15做一对比可见，先序遍历序列与后序遍历序列并非简单的逆序关系。

■ 中序遍历

再次仿照以上模式，可实现递归版中序遍历算法如代码5.13所示。

```
1 template <typename T, typename VST> //元素类型、操作器
2 void travIn_R(BinNodePosi(T) x, VST& visit) { //二叉树中序遍历算法（递归版）
3     if (!x) return;
4     travIn_R(x->lChild, visit);
5     visit(x->data);
6     travIn_R(x->rChild, visit);
7 }
```

代码5.13 二叉树中序遍历算法（递归版）

按照中序遍历规则，为遍历非空的（子）树 x ，将依次递归遍历其左子树、访问节点 x 、递归遍历其右子树。以上二叉树实例的中序遍历过程以及生成的遍历序列，如图5.17所示。

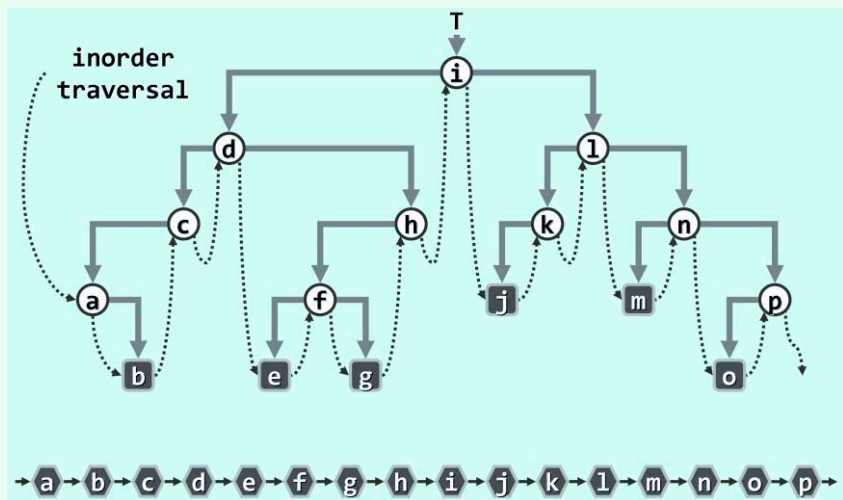


图5.17 二叉树的中序遍历序列

与以上的先序和后序遍历序列做一对比不难发现，各节点在中序遍历序列中的局部次序，与按照有序树定义所确定的全局左、右次序完全吻合。这一现象并非巧合，在第7章和第8章中，这正是搜索树及其等价变换的原理和依据所在。

5.4.2 *迭代版先序遍历

无论以上各种递归式遍历算法还是以下各种迭代式遍历算法，都只需渐进的线性时间（习题[5-9]和[5-11]）；而且相对而言，前者更加简明。既然如此，有何必要介绍迭代式遍历算法呢？

首先，递归版遍历算法时间、空间复杂度的常系数，相对于迭代版更大。同时，从学习的角度来看，从底层实现迭代式遍历，也是加深对相关过程与技巧理解的有效途径。

■ 版本1

观察先序遍历的递归版（代码5.11）可发现，其中针对右子树的递归属于尾递归，左子树的则接近于尾递归。故参照消除尾递归的一般性方法，不难将其改写迭代版（习题[5-10]）。

■ 版本2

很遗憾，以上思路并不容易推广到非尾递归的场合，比如在中序或后序遍历中，至少有一个递归方向严格地不属于尾递归。此时，如下另一迭代式版本的实现思路，则更具参考价值。

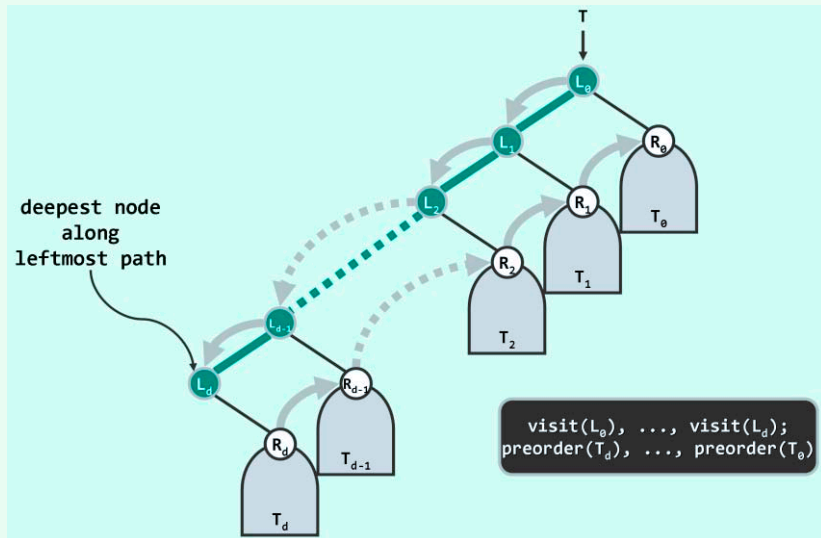


图5.18 先序遍历过程：先沿最左侧通路自顶而下访问沿途节点，再自底而上依次遍历这些节点的右子树

如图5.18所示，在二叉树T中，从根节点出发沿着左分支一直下行的那条通路（以粗线示意），称作最左侧通路（leftmost path）。若将沿途节点分别记作 L_k ， $k = 0, 1, 2, \dots, d$ ，则最左侧通路终止于没有左孩子末端节点 L_d 。若这些节点的右孩子和右子树分别记作 R_k 和 T_k ， $k = 0, 1, 2, \dots, d$ ，则该二叉树的先序遍历序列可表示为：

```
preorder(T) =
    visit(L0),    visit(L1), ...,    visit(Ld);
    preorder(Td), ...,    preorder(T1), preorder(T0)
```

也就是说，先序遍历序列可分解为两段：沿最左侧通路自顶而下访问的各节点，以及自底而上遍历的对应右子树。基于对先序遍历序列的这一理解，可以导出以下迭代式先序遍历算法。

```
1 //从当前节点出发，沿左分支不断深入，直至没有左分支的节点；沿途节点遇到后立即访问
2 template <typename T, typename VST> //元素类型、操作器
3 static void visitAlongLeftBranch(BinNodePosi(T) x, VST& visit, Stack<BinNodePosi(T)>& S) {
4     while (x) {
5         visit(x->data); //访问当前节点
6         S.push(x->rChild); //右孩子入栈暂存（可优化：通过判断，避免空的右孩子入栈）
7         x = x->lChild; //沿左分支深入一层
8     }
9 }
```

```
10
11 template <typename T, typename VST> //元素类型、操作器
12 void travPre_I2(BinNodePosi(T) x, VST& visit) { //二叉树先序遍历算法（迭代版#2）
13     Stack<BinNodePosi(T)> S; //辅助栈
14     while (true) {
15         visitAlongLeftBranch(x, visit, S); //从当前节点出发，逐批访问
16         if (S.empty()) break; //直到栈空
17         x = S.pop(); //弹出下一批的起点
18     }
19 }
```

代码5.14 二叉树先序遍历算法（迭代版#2）

如代码5.14所示，在全树以及其中每一棵子树的根节点处，该算法都首先调用函数 `VisitAlongLeftBranch()`，自顶而下访问最左侧通路沿途的各个节点。这里也使用了一个辅助栈，逆序记录最左侧通路上的节点，以便确定其对应右子树自底而上的遍历次序。

5.4.3 *迭代版中序遍历

如上所述，在中序遍历的递归版本（126页代码5.13）中，尽管右子树的递归遍历是尾递归，但左子树绝对不是。实际上，实现迭代式中序遍历算法的难点正在于此，不过好在迭代式先序遍历的版本2可以为我们提供启发和借鉴。

■ 版本1

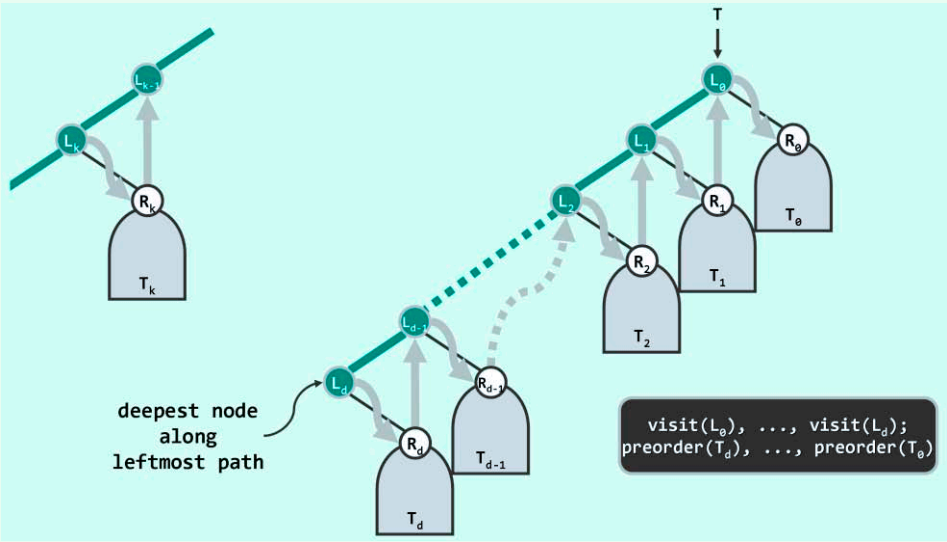


图5.19 中序遍历过程：
顺着最左侧通路，自底而上依次访问沿途各节点及其右子树

如图5.19所示，参照迭代式先序遍历版本2的思路，再次考查二叉树T的最左侧通路，并对其中的节点和子树标记命名。于是，T的中序遍历序列可表示为：